# Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead

JIEKE SHI, ZHOU YANG, and DAVID LO, School of Computing and Information Systems, Singapore Management University, Singapore, Singapore

Large Language Models (LLMs) have recently shown remarkable capabilities in various software engineering tasks, spurring the rapid growth of the Large Language Models for Software Engineering (LLM4SE) area. However, limited attention has been paid to developing efficient LLM4SE techniques that demand minimal computational cost, time, and memory resources, as well as green LLM4SE solutions that reduce energy consumption, water usage, and carbon emissions.

This article aims to redirect the focus of the research community toward the efficiency and greenness of LLM4SE, while also sharing potential research directions to achieve this goal. It commences with a brief overview of the significance of LLM4SE and highlights the need for efficient and green LLM4SE solutions. Subsequently, the article presents a vision for a future where efficient and green LLM4SE revolutionizes the LLM-based software engineering tool landscape, benefiting various stakeholders, including industry, individual practitioners, and society. The article then delineates a roadmap for future research, outlining specific research paths and potential solutions for the research community to pursue. While not intended to be a definitive guide, the article aims to inspire further progress, with the ultimate goal of establishing efficient and green LLM4SE as a central element in the future of software engineering.

CCS Concepts: • **General and reference** → **Surveys and overviews**; • **Software and its engineering** → **Software development techniques**; • **Computing methodologies** → **Artificial intelligence**;

Additional Key Words and Phrases: Software Engineering, Large Language Models, Efficiency, Greenness

**ACM Reference format:**
Jieke Shi, Zhou Yang, and David Lo. 2025. Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead. *ACM Trans. Softw. Eng. Methodol.* 34, 5, Article 137 (May 2025), 22 pages.
https://doi.org/10.1145/3708525

## 1 Introduction

Recent advances in **large language models (LLMs)** have spurred their widespread adoption in various fields, notably within software engineering [8, 27, 34, 48, 131, 152, 156]. Trained on massive amounts of data, LLMs have shown their prowess in processing and generating code written in

This research project was supported by the National Research Foundation, under its Investigatorship Grant (NRF-NRFI08-2022-0002). Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not reflect the views of National Research Foundation, Singapore.
Authors' Contact Information: Jieke Shi, School of Computing and Information Systems, Singapore Management University, Singapore, Singapore; e-mail: jiekeshi@smu.edu.sg; Zhou Yang (corresponding author), School of Computing and Information Systems, Singapore Management University, Singapore, Singapore; e-mail: zyang@smu.edu.sg; David Lo, School of Computing and Information Systems, Singapore Management University, Singapore, Singapore; e-mail: davidlo@smu.edu.sg.


ACM 1557-7392/2025/5-ART137
https://doi.org/10.1145/3708525

various programming languages, with exceptional performance across a spectrum of code-related tasks, ranging from code generation [13, 15, 143] and summarization [2, 3, 71] to vulnerability detection [97, 120, 159], program repair [57, 59, 142], and more [29, 90, 102, 106, 153]. This success has fueled the rapid growth of the **large language models for software engineering (LLM4SE)** area, which has recently become one of the most active areas in software engineering.

To date, certain LLM4SE techniques have been translated into real-world applications. For instance, GitHub Copilot [33], powered by OpenAI's GPT-4 [92], assists developers in writing code, while Google VirusTotal Code Insight [103] leverages the Sec-PaLM model [18] for malware analysis. These tools have introduced a new level of automation to software development, enhancing both developer productivity [19, 60] and software quality [98, 127]. However, due to the computationally-intensive and energy-demanding nature of LLMs [46, 69, 101], most of these LLM4SE applications are facilitated by large companies with abundant computational resources and energy supplies that are not readily available to the broader software engineering community, including startups and individual developers. To democratize LLM4SE techniques and make them beneficial to all software engineering practitioners, several key issues must be addressed, particularly their unsatisfactory efficiency (i.e., high computational cost, memory usage, and time consumption) and lack of greenness (i.e., high energy usage, water consumption, and carbon emissions) [72, 144].

Our research community has recently acknowledged the growing need to enhance LLM4SE techniques' efficiency and greenness, with several studies making strides for this purpose, such as those by Shi et al. [107, 108] and Sun et al. [114, 115]. However, this area remains far from fully explored, with many breakthroughs yet to be made, and the overall landscape still unclear as we look toward 2030 and beyond. This article, therefore, presents our vision for the future of efficient and green LLM4SE, identifies research gaps and potential opportunities, and outlines a roadmap for our research community to advance this area. Our vision is encapsulated in the following statement:

> *"By 2030, LLM4SE tools will be efficient, green, more affordable, and more accessible to individual software practitioners, while fostering sustainability and reducing the environmental impact across the software industry and society."*

The rest of the article is organized as follows. In Section 2, we define efficient and green LLM4SE and highlight its importance. Section 3 then reviews on the current state of LLM4SE solutions in terms of efficiency and greenness, followed by a vision for the future of efficient and green LLM4SE in Section 4. We provide a roadmap outlining specific research paths and potential solutions that the research community can pursue in Section 5. Finally, we conclude with a summary of the key points and a call to action for the research community to embrace efficient and green LLM4SE techniques in Section 6.

## 2 Preliminaries

### 2.1 LLMs

LLMs refer to a class of neural network models that leverage the Transformer architecture [126] to process and generate language text. The term "large" signifies the vast number of parameters in these models, typically ranging from hundreds of millions to billions. Here, we briefly introduce the mainstream LLMs that have driven the rapid progress in the LLM4SE area. Following Hou et al. [48], we categorize these models into three groups based on their architectures: (1) encoder-only, (2) encoder–decoder, and (3) decoder-only LLMs. We also refer readers to recent surveys [48, 145] for a more comprehensive overview of LLMs.

*2.1.1 Encoder-Only LLMs.* Encoder-only LLMs, such as BERT [24] and RoBERTa [80], utilize only the encoder component of the Transformer architecture [126] to process input and capture

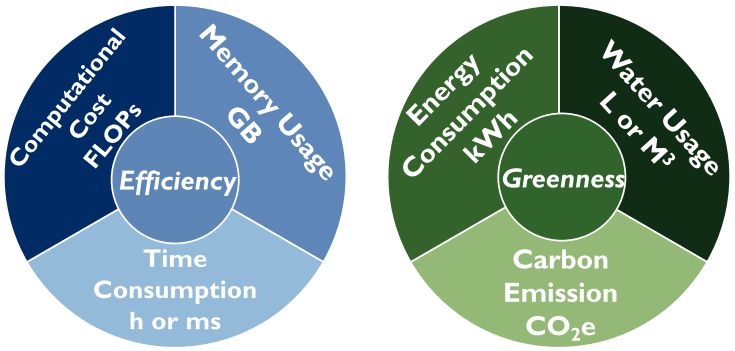


Fig. 1. Six Dimensions of efficient and green LLM4SE, along with their metrics.

contextual information for downstream tasks. In the software engineering domain, CodeBERT [30] and GraphCodeBERT [41] are prominent examples of encoder-only LLMs tailored for programming languages. Additional models developed for various SE tasks include CuBERT [61], CCBERT [158], BERTOverflow [119], SOBERT [44], CodeRetriever [74], FAIR [90], UniXcoder [40], and VarBERT [93].

*2.1.2 Encoder–Decoder LLMs.* Encoder–decoder LLMs are equipped with both the encoder and decoder components of Transformer [126], which can respectively process input and generate output text. These LLMs are more versatile than encoder-only models in that they can better handle a wide range of generation tasks. Models such as PLBART [1], CodeT5 [134], CodeT5+ [132], AlphaCode [75], SPT-Code [89], CoTexT [96], NatGen [11], CoditT5 [151], and GrammarT5 [160] showcase the effectiveness of encoder–decoder LLMs in various SE tasks.

*2.1.3 Decoder-Only LLMs.* Decoder-only LLMs, currently the most prevalent type of LLMs, are exemplified by OpenAI's GPT series [9, 92]. These models rely solely on the decoder component of the Transformer and are designed for autoregressive generation tasks, where they predict the next token in a sequence based on the preceding tokens. Numerous decoder-only LLMs tailored for code generation have been developed, including CodeGPT [84], GPT-C [117], Codex [15], PolyCoder [143], CodeGen [87, 88], Code Llama [99], StarCoder [73, 82], and Magicoder [137].

## 2.2 Definition of Efficient and Green LLM4SE

As illustrated in Figure 1, this article defines efficient and green LLM4SE techniques as those that minimize six key dimensions from training to deployment: computational cost, memory usage, time consumption for efficiency, energy consumption, water usage, and carbon emissions for greenness. These dimensions and their metrics are described as follows.

*Dimensions of Efficiency.* Computational cost refers to the total arithmetic operations (e.g., addition, multiplication) required for training or inference in LLM4SE techniques, typically measured in **floating-point operations (FLOPs)**, as seen in recent studies [107, 108]. Memory usage measures the amount of memory consumed during training or deployment, commonly expressed in gigabytes (GB). Time consumption accounts for the duration needed for training or the latency for inference, typically quantified in hours (h) for training and milliseconds (ms) for inference [107, 108, 136].

*Dimensions of Greenness.* The term "green" is often used interchangeably with "sustainable" in the literature. However, recent studies [39, 56], including work by Calero et al. [10] and Cruz et al. [20], have called for a distinction between the two, with "green" specifically referring to environmental

sustainability, while "sustainable" covers a broader spectrum, including social and economic dimensions. In this article, we focus on the environmental aspect of LLM4SE techniques and use the term "green"; other aspects of sustainability are beyond our scope.

We define green LLM4SE techniques as those with low energy consumption, water usage, and carbon emissions. Energy consumption refers to the electrical power required during both training and inference phases, typically measured in **kilowatt-hours (kWh)** [107]. Water usage denotes the volume of water used for electricity generation or server cooling, often measured in liters (L) or cubic meters ($m^3$) [72]. Carbon emissions account for the release of greenhouse gases, primarily carbon dioxide ($CO_2$), during training or inference, typically measured in kilograms (kg) or tons (t) of **$CO_2$ equivalents ($CO_2$e)** [26, 85, 95].

We acknowledge that these dimensions and metrics may not be exhaustive, but they offer an overview for evaluating the efficiency and environmental impact of LLM4SE methods. Building on this, we discuss the significance of developing efficient and green LLM4SE techniques and their synergy in the following subsection.

### 2.3 Significance of Efficient and Green LLM4SE

*2.3.1 Why Efficient LLM4SE?* Most current LLM4SE techniques rely on large-scale models like Code Llama [99], which incur significant computational costs, requiring vast amounts of FLOPs for both training and inference. According to estimates from the open source community [21], LLMs with architectures similar to Code Llama and LLaMA 2 [123] demand over 100 TeraFLOPs for inference, surpassing the capabilities of many older consumer-grade GPUs [141]. Running these computationally-intensive LLMs on unsuitable hardware not only results in slow performance, but also risks overheating, potentially damaging the equipment.

Moreover, developing LLM4SE techniques typically involves a prolonged training process of millions of GPU hours [88, 99, 122]. This lengthy training, coupled with the subsequent operation of LLMs, necessitates substantial computational resources, including thousands of high-performance GPUs [43] and thousands of GB of memory, translating into high financial costs. For example, training OpenAI's GPT-3 [9] costs over $4 million [66]. Such expenses are prohibitive for many software practitioners, particularly those in smaller companies or developing countries, compelling them to rely on cloud services offered by large or well-funded companies. However, this reliance also brings significant expenses, peaking at $200,000 per month [66], and raises privacy concerns due to the necessity of sharing sensitive data with service providers [81, 145].

Even aside from the costs and privacy issues, users often face a suboptimal experience with cloud-hosted LLM4SE techniques due to prolonged response time caused by network delays [32, 107, 108, 136]. While deploying LLMs on users' devices can address issues of poor user experience from network delays and enhance data privacy [81, 107, 108], accomplishing this task remains challenging without efficient LLM4SE techniques, as most current LLMs demand substantial memory beyond what many personal devices, such as laptops, can accommodate [108, 136]. Even when memory is sufficient, high inference latency—sometimes lasting several seconds [108, 136]—and the significant energy consumption of LLMs can drain the battery of personal devices, like laptops, within an hour, making them unsuitable for prolonged use [107, 136]. Building efficient LLM4SE techniques can address the above challenges and make LLM4SE techniques more accessible to all software engineering practitioners, regardless of their computational resources.

*2.3.2 Why Green LLM4SE?* Training LLMs typically requires vast amounts of energy, resulting in high carbon emissions. For example, training LLaMA [122] consumes 2,638,000 kWh of electricity, equivalent to the annual electricity usage of 445 citizens of Denmark, and emits 1,015 tons of $CO_2$e, comparable to the annual emissions of 221 citizens of Denmark [54]. The energy

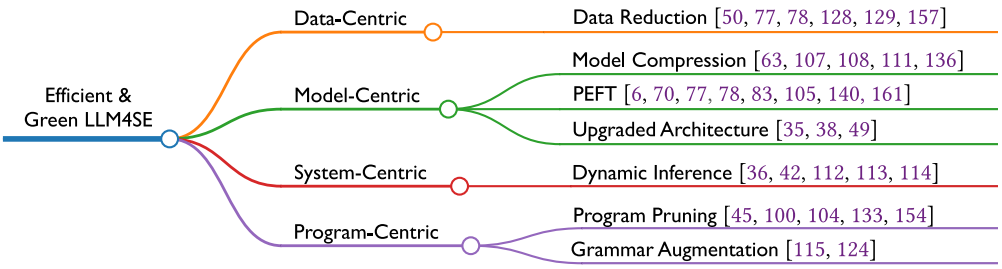


Fig. 2. Current techniques for efficient and green LLM4SE.

consumption of LLM inference is also considerable; each ChatGPT inference consumes 2.9 **watt-hours (Wh)** of electricity, nearly 10 times the 0.3 Wh consumption of a single Google search [22, 109]. Moreover, to support these energy-hungry models, data centers and cloud computing facilities—major contributors to global carbon emissions—are heavily relied upon [7, 139]. These facilities also require enormous amounts of water for cooling and electricity generation, further straining the environment [72]. In 2022, the combined water usage of data centers powering AI models from companies like Google, Microsoft, and Meta was estimated to be 2.2 billion $m^3$, nearly double Denmark's annual water consumption [72]. This high energy consumption, water use, and carbon output not only make LLM4SE techniques expensive to develop and maintain but also contribute significantly to climate change and environmental degradation. Therefore, developing more environmentally friendly, green LLM4SE techniques is crucial to mitigate these negative environmental impacts.

*2.3.3 Why Synergizing Efficient and Green LLM4SE?* Efficient and green LLM4SE techniques are closely connected. Shi et al.'s recent study [107] shows that reducing the memory footprint of LLMs can decrease their energy consumption, while both their work and that of Wei et al. [136] also demonstrate that green methods aimed at reducing energy consumption and carbon emissions, such as knowledge distillation and quantization, can also improve the efficiency of LLMs by reducing inference latency and memory usage. These cases illustrate that improving efficiency in LLM4SE techniques can lead to upgraded greenness, and *vice versa*. However, efficiency and greenness, though related, are not synonymous, meaning that achieving one does not always guarantee the full optimization of the other. For instance, Shi et al. [108] propose an efficient solution by compressing LLMs, but their compressed models still consume significant energy and have unsatisfactory carbon emissions, indicating room for improvement in their environmental impact, which is addressed in their subsequent work [107]. Therefore, we advocate for a synergistic approach that integrates both efficiency and greenness in LLM4SE techniques, achieving the best of both worlds to make LLM4SE techniques both accessible and environmentally sustainable.

## 3 Literature Review

In this section, we review the literature and provide a brief summary of the current state of efficient and green LLM4SE techniques. As shown in Figure 2, we categorize the literature into four main perspectives—data-centric, model-centric, system-centric, and program-centric—based on their methodological focus, and further organize them into specific techniques within each category.

### 3.1 Data-Centric Efficient and Green LLM4SE

The data-centric techniques focus on reducing or optimizing the data required to train LLMs, thereby improving the efficiency and greenness of LLM4SE techniques.

Current data-centric techniques for efficient and green LLM4SE primarily focus on data reduction, which seeks to minimize the amount of data needed to train LLMs, thereby reducing training time, energy consumption, water usage, and carbon emissions. One approach in this direction is active learning, as explored by Hu et al. [50], which apply active learning to CodeBERT and GraphCodeBERT to reduce the size of the fine-tuning data, finding that using less than 10% of the data significantly degrades LLM performance in code summarization. In contrast, studies [77, 78, 128] have demonstrated that **parameter-efficient fine-tuning (PEFT)** methods are more effective with limited data. Notably, Liu et al. [77] show that PEFT can achieve competitive performance in code clone detection with just 1,000 labeled examples. Wang et al. [129] propose a curriculum learning strategy for training CodeBERT and GraphCodeBERT, where examples are presented in a structured order from simple to complex. This strategy allows their models, which use only 10% of the training data for code clone detection, to outperform state-of-the-art LLMs trained on the full dataset. More recently, Zhou et al. [157] propose refining and filtering training data for large LLMs using a smaller LLM. By retaining no more than 30% of the original training data, they achieve up to 20 times lower computational costs in training LLMs like Code Llama, with comparable performance in tasks like code generation and mathematical reasoning.

### 3.2 Model-Centric Efficient and Green LLM4SE

Model-centric techniques focus on optimizing the LLMs themselves, including model parameters and architecture, to improve efficiency and greenness.

*3.2.1 Model Compression.* Model compression aims to reduce the size of a model, thereby optimizing other relevant metrics such as inference latency, memory usage, and energy consumption. As the pioneering work in compressing code LLMs, Shi et al. [108] propose Compressor, which uses a genetic algorithm-based method for simplifying models and applies knowledge distillation to compress LLMs to 3 **megabytes (MB)**. Their compressed LLMs improve inference latency by 4.23× on average. Subsequently, Shi et al. [107] also present Avatar, which finds pareto-optimal compressed LLMs that balance model size, inference latency, energy consumption, and carbon footprint, instead of solely focusing on model size like Compressor. The models produced by Avatar significantly reduce energy consumption (up to 184× less), carbon footprint (up to 157× less), and inference latency (up to 76× faster). Additionally, Su and McMillan [111] use knowledge distillation on GPT-3.5 to obtain a smaller and more efficient model for code summarization. Wei et al. [136] and Kaushal et al. [63] use quantization and low-rank decomposition to compress LLMs for code generation, respectively, both achieving significant efficiency improvements.

*3.2.2 PEFT.* PEFT aims to improve the training efficiency of LLMs. During training, it updates only a small set of LLM parameters while freezing the rest, thereby greatly reducing computational and memory costs. Researchers have applied PEFT to LLMs for various software engineering tasks [77, 78, 105], such as code generation [140, 161], code review [83], clone detection [6], and program repair [70]. These studies show that PEFT can significantly reduce the training time and memory consumption of LLMs.

*3.2.3 Upgraded Architecture.* This line of research focuses on improving the architecture of LLMs to enhance their efficiency. For instance, Hu et al. [49] propose a novel mechanism in CodeBERT to split long code into short blocks, significantly accelerates the processing of long codes, improving efficiency by over 5× in code search compared to the original CodeBERT. Gotmare et al. [35] introduce an efficient text-to-code search framework that utilizes cascaded fast and slow LLMs, where a fast model performs rapid retrieval, followed by a slower, larger model for re-ranking to improve accuracy. This framework strikes an optimal balance between efficiency and effectiveness in

code search. Additionally, Gu et al. [38] present CoSHC, a method designed to accelerate CodeBERT through deep hashing. By leveraging hash codes for code representations generated by CodeBERT, their method reduces retrieval time by over 90% compared to the original CodeBERT model.

### 3.3 System-Centric Efficient and Green LLM4SE

System-centric techniques focus on optimizing various aspects of the system or pipeline, such as the inference process or decoding strategy, to enhance efficiency and greenness. We list system-centric techniques separately inspired by Miao et al. [86].

The current techniques in this category mainly focus on applying dynamic inference, which manipulates the inference process of LLMs to improve model efficiency. An early work by Sun et al. [113, 114] introduces an early rejection mechanism to reject prompts that cannot generate useful completions during inference. This mechanism saves 23.3% of the computational cost of LLMs in code completion. After that, Sun et al. [112] discover that 54.4% of tokens can be accurately generated using only the first layer of the GPT-2 model. They develop a method capable of skipping an average of 1.7 layers out of 16 in the models, resulting in a speedup of 11.2% in code completion. Similarly, Grishina et al. [36] show that only 3 of the 12 layers of CodeBERT are sufficient for vulnerability detection with a 3.3× speedup in fine-tuning. Recently, Guo et al. [42] introduce CodeFast, a method that terminates the inference process early when excess tokens are unnecessary. This is achieved using a lightweight model called GenGuard, which predicts whether to halt inference at each step. Their method significantly improves the inference speed of various LLMs in code generation, ranging from 34% to 452%, without compromising the quality of generated code.

### 3.4 Program-Centric Efficient and Green LLM4SE

Program-centric techniques aim to optimize the input programs fed into LLMs to enhance efficiency and greenness. These methods often focus on reducing the number of tokens or restructuring the grammar of input programs, leading to faster processing and lower resource consumption.

*3.4.1 Program Pruning.* Pruning the tokens of input programs can improve the efficiency of LLMs, since their inference time is affected by the input length. Zhang et al. [154] introduce DietCode, which identifies statements and tokens with the highest attention values, resulting in a 40% reduction in the computational cost of CodeBERT inference. In addition, Hidvégi et al. [45] optimize prompts to reduce token cost by 62% in program repair tasks. Shi et al. [104] simplify input for LLMs, resulting in up to 40% reduction in inference time. Recently, Saad et al. [100] propose a method to prune unimportant tokens according to the attention scores when token sequences go through the layers of the LLMs, achieving over 58% reduction memory consumption and 51% improvements in model throughput. Wang et al. [133] introduce SlimCode, which removes redundant tokens by leveraging the structural information of code, such as the **abstract syntax tree (AST)** and Program Dependency Graph. This approach improves effectiveness by up to 9% in code search and summarization, and is up to 133 times faster compared to DietCode.

*3.4.2 Grammar Augmentation.* Grammar augmentation aims to enhance the efficiency of LLMs by restructuring input programs to reduce token usage, thereby improving inference time. Sun et al. [115] introduce SimPy, the first AI-oriented grammar for Python. SimPy eliminates redundant tokens in Python code, such as unnecessary whitespace and semicolons, using heuristic grammar rules, while preserving the original AST structure. This approach reduces token usage by 13.5% and improves performance by 10.4% in code-related tasks for models like Code Llama and GPT-4. Additionally, Ugare et al. [124] propose SynCode, which integrates a context-free grammar into the decoding process of LLMs. This method forces the model to efficiently generate syntactically valid

code, accelerating the decoding process by up to 19% when generating syntactically correct code snippets.

In addition to the above techniques, several studies do not provide a specific technique but focus on the efficiency and greenness of the code generated by LLM4SE techniques, such as [51, 79, 125]. While these studies make valuable contributions to the LLM4SE area, they are not included in the above categories due to their primary focus on the generated code rather than the LLMs themselves, which is beyond the scope of our literature review.

Additionally, several studies focus on the greenness of machine learning/deep learning models outside the LLM4SE domain, such as [4, 25, 53, 62, 150]. Among these, Durán et al. [25] analyze ML serving architectural design decisions and their impact on energy efficiency. Kannan et al. [62] introduce GreenRunner, a tool that selects models based on a natural language description of the task, with an emphasis on energy efficiency. Alizadeh and Castor [4] examine the energy efficiency and inference performance of DL runtime infrastructures across frameworks such as PyTorch and TensorFlow. Husom et al. [53] propose CEMAI, a tool designed to monitor and analyze carbon emissions across the entire lifecycle of ML model development. Yuan et al. [150] empirically assess whether knowledge distillation can improve the energy efficiency of LLMs in natural language processing domains without compromising performance. These studies are excluded from our review, as they do not specifically address the efficiency and greenness of LLM4SE techniques. However, their findings could be valuable for future research in this area. For example, adapting the GreenRunner tool of Kannan et al. [62] to select energy-efficient LLMs for software engineering tasks may be beneficial, and leveraging the architectural design decisions of Durán et al. [25] potentially helps optimize the energy efficiency of LLM4SE techniques. We encourage future research to explore these possibilities.

**Reflection** — *The literature review highlights remarkable progress in developing efficient and green LLM4SE techniques across data, model, system, and program-centric perspectives. However, this area remains far from fully explored, with more breakthroughs needed to address the efficiency and greenness challenges of LLM4SE.*

## 4 Future: LLM4SE of 2030 and Beyond

After reviewing the current state of efficient and green LLM4SE solutions, we now turn our attention to the future. According to the current trends and the potential of our research community, we envision a future where efficient and green LLM4SE solutions revolutionize the LLM-based software engineering tool landscape, benefiting various stakeholders, including industry, individual practitioners, and society.

### 4.1 For Individual Software Practitioners

The advancement of efficient and green LLM4SE can pave the way for the emergence and widespread adoption of *Private, Personalized, Trusted, and Collaborative Software Engineering Assistants*. These new assistants will transcend current tools such as GitHub Copilot [33], which are confined to one single task like code suggestion, rely on cloud infrastructure, pose potential privacy risks [91, 146], and are challenging or costly to customize [17]. With minimal computational resources and energy consumption, these new assistants can be seamlessly integrated into software practitioners' local development environments. They remain entirely private to the practitioners, securing their intellectual property and sensitive information from various threats [23, 52, 91, 146]. This integration facilitates real-time interaction between software practitioners and their assistants with minimal latency, enabling them to provide instant feedback and suggestions to practitioners as they write

code, test software, or engage in other software engineering activities. Such real-time interactions can occur without an Internet connection, which is particularly crucial in regions of developing countries with limited access to high-speed Internet. Furthermore, this interaction can consume less computational resources and energy than today, making it financially viable for individual practitioners to utilize these assistants on personal devices like laptops.

Moreover, the assistants would deliver personalized assistance for various software engineering tasks, tailored to individual preferences and project requirements. Such personalization benefits from the efficient and green nature of the assistants, allowing it to be accomplished with minimal computational resources and energy consumption. Following such personalization, the assistants will furnish more precise and pertinent assistance to software practitioners, thereby catalyzing trust between software practitioners and their assistants. This trust will enhance the collaboration between software practitioners and assistants, enabling them to work together more effectively to improve productivity, software quality, and the overall development experience.

### 4.2 For Industry

Advances in efficient and green LLM4SE will facilitate the development of *Low-cost and Low-latency LLM-based Software Engineering Tools*. The existing LLM-based software engineering tool landscape is dominated by heavyweight solutions that are computationally expensive and energy-intensive. Companies must not only invest in large amounts of computing resources such as GPUs to develop and run these tools, but also have to bear the high cost of the associated energy consumption (electricity, cooling, etc.) and carbon emissions, requiring the purchase of many carbon credits to compensate for the environmental impact [65]. These costs limit the existing tools' profitability. For example, Microsoft's GitHub Copilot currently loses $20 per user per month instead of making a profit [147]. Moreover, these tools often have high latency, often resulting in a poor user experience [32, 136].

By adopting efficient and green LLM4SE solutions, companies can replace existing imperfect tools with upgraded ones that are faster, greener, and require fewer resources to develop and operate. Not only can these upgraded tools be used internally by companies to streamline their software development processes, but they can also be offered as Software as a Service solutions to other companies and developers, giving them access to cutting-edge LLM-based software engineering tools at reduced costs and faster response times, all without requiring a large investment in computing resources. In addition, efficient and green LLM4SE solutions can diminish the entry barriers for LLM-based software engineering tools, presenting significant opportunities for **small and medium enterprises (SMEs)** and startups, which typically face resource constraints while striving to improve their development workflows. This fosters a more equitable environment in the software industry, allowing SMEs and startups to compete on par with larger companies. Overall, for the software industry, adopting efficient and green LLM4SE will revolutionize the LLM-based software engineering tool landscape, making it more affordable and accessible to all stakeholders within the industry.

### 4.3 For Society

Efficient and green LLM4SE will play a critical role in fostering *Better Environmental Sustainability in Software Engineering*. The software industry is a significant contributor to global carbon emissions and energy consumption, with data centers and cloud computing facilities, often used for LLM development and operations, being major responsible parties [7, 139]. Embracing efficient and green LLM4SE solutions can mitigate these environmental impacts. These new solutions promise to reduce the energy consumption and carbon emissions associated with LLM development and operation, thereby reducing reliance on data centers and cloud computing facilities and promoting greener practices in the software industry. Transitioning to efficient and green LLM4SE will

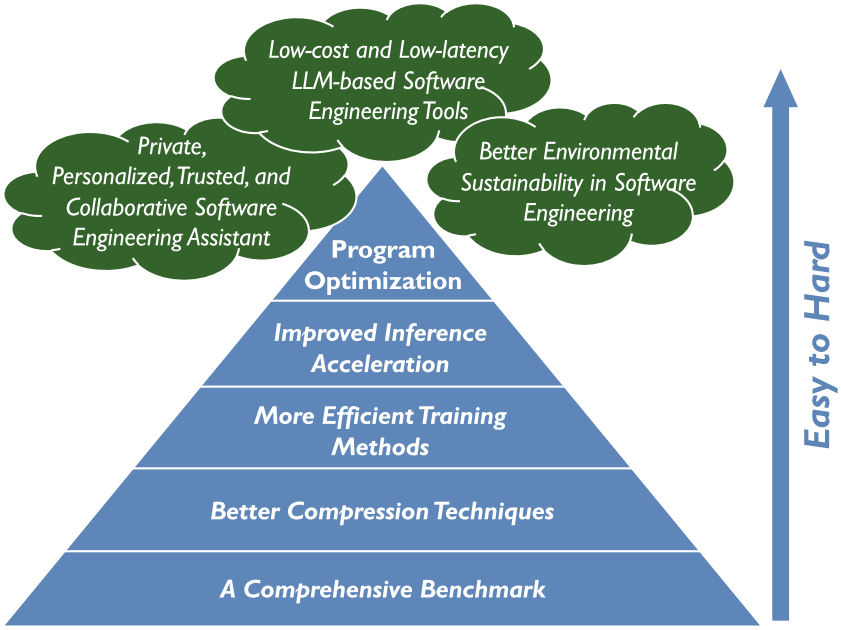


Fig. 3. Roadmap for efficient and green LLM4SE.

not only benefit the environment, but will also align with the growing societal emphasis on environmental sustainability. Large technology companies can achieve sustainability goals such as carbon neutrality [116, 130], while smaller companies and individual developers can share more carbon credits, facilitating their rapid growth [65]. By promoting efficient and green LLM4SE, the software industry can demonstrate its commitment to environmental responsibility and contribute to a more sustainable future for all.

**Vision** — *By 2030, the future landscape of LLM4SE tools will feature low-cost assistants integrated into local development environments, offering real-time support to practitioners, while industry-wide adoption of greener tools will reduce environmental impact, promote sustainability, and ensure equitable access for all stakeholders, benefiting both the software industry and society.*

## 5 The Road Ahead

This section presents a roadmap for achieving efficient and sustainable LLM4SE, outlining several specific research paths and potential techniques for further exploration by the research community. It is important to note that some research paths may overlap or be interrelated, and may not fit exclusively within the categories of data, model, system, or program-centric approaches discussed in Section 3. Therefore, the roadmap is presented holistically, with each research topic or technique potentially addressing multiple dimensions. Figure 3 illustrates this roadmap for efficient and green LLM4SE. Based on the research gaps identified in Section 3, we sort the roadmap's difficulty level from low to high, according to the current maturity of that research areas. If a research path has fewer studies or is more challenging to address, we classify it as high difficulty.

### 5.1 A Comprehensive Benchmark

Prior to embarking on the development of LLM4SE techniques, it is essential to establish a comprehensive benchmark dataset to evaluate their efficiency and greenness. While many existing benchmarks, such as CodeXGLUE [84] and CoderEval [149], evaluate the effectiveness (e.g., accuracy) of LLMs in certain software engineering tasks, very few specifically target the efficiency and greenness of LLM4SE. A few studies have collected datasets to evaluate the efficiency or energy consumption of the code generated by LLMs, such as EffiBench [51], and the datasets constructed by Liu et al. [79] and Vartziotis et al. [125]. Nevertheless, these datasets primarily focus on evaluating the generated code rather than the LLMs, and thus are not comprehensive enough. Consequently, researchers and practitioners lack a comprehensive understanding of the current landscape of LLM4SE development in terms of efficiency and greenness, making it difficult for them to compare different LLM4SE techniques and make informed decisions about which one to adopt. We suggest that the research community prioritize curating a diverse set of tasks and datasets representative of real-world software engineering applications to benchmark the efficiency and greenness of LLM4SE techniques. These efforts will lay the foundation for future advances in the area.

### 5.2 More Efficient Training Methods

Training LLMs is computationally intensive and time-consuming, posing a significant bottleneck in the development of LLM4SE. While some studies have leveraged PEFT-based methods to accelerate LLM training, as discussed in Section 3, more efficient training methods, such as data and model parallelism [155], pipeline parallelism [64], and improved optimizers [148], have not been thoroughly explored for LLM4SE. Data parallelism involves distributing data across multiple devices, with each device holding a full copy of the model, while model parallelism splits the model across devices, with each device holding only a portion. Both methods can accelerate LLM training when multiple GPUs are available. Pipeline parallelism combines data and model parallelism, making it suitable for training very large LLMs. Additionally, You et al. [148] proposed a novel optimizer that automatically adjusts the learning rate for LLMs' each layer and batch size, enabling the use of very large batch sizes for faster convergence of LLMs. This optimizer allowed for pre-training a BERT model in just 76 minutes back in 2019, without relying on large GPUs. However, no study has yet systematically evaluated these methods in the context of LLM4SE. We suggest that researchers apply and assess these approaches to identify the most effective techniques and the unique challenges in developing LLM4SE, ultimately paving the way for new training acceleration methods tailored specifically for LLM4SE.

Furthermore, **retrieval-augmented generation (RAG)** [68] could be explored as a way to reduce the cost of specializing LLMs for software engineering tasks. RAG retrieves texts that semantically match a query from an external knowledge base and passes them to an LLM to extract the correct answer, allowing LLMs to leverage existing knowledge to generate domain-specific responses without requiring extensive retraining [28]. While recent studies have demonstrated RAG's effectiveness in software engineering tasks such as code suggestion [13, 135] and summarization [94], its primary benefit for LLM4SE lies in eliminating the need for fine-tuning or training LLMs from scratch. However, the inference cost of RAG remains high, even surpassing that of standalone LLMs due to the additional queries made to the knowledge base [47]. Therefore, we recommend that researchers explore ways to optimize RAG's inference cost for LLM4SE, such as using more efficient retrieval methods [47, 55] or caching retrieved knowledge to reduce the number of queries to the knowledge base [58].

### 5.3 Better Compression Techniques

Popular model compression techniques, including knowledge distillation and quantization, have shown promise in compressing LLMs for efficient and green LLM4SE, as introduced in Section 3. However, further exploration is still needed to optimize the efficiency of LLMs to a satisfactory extent, aiming at response times in the range of a few tens of milliseconds [5] and memory consumption of less than 50 MB [118]. Current compression techniques may fall short of these requirements, especially for very large LLMs like Code Llama [99], which can range from tens to hundreds of GB in size. Therefore, there is a pressing need to propose novel compression techniques tailored to these very large LLMs in order to achieve the desired efficiency. Furthermore, while knowledge distillation has been the primary focus of current compression efforts for LLM4SE techniques [107, 108, 111], other techniques like quantization and pruning have received relatively less attention, with fewer studies available. Hence, we advocate for increased research efforts to explore the potential of quantization and pruning techniques alongside the innovation of new knowledge distillation methods. Moreover, combining these techniques can also be investigated to achieve better efficiency and greenness improvements for LLM4SE techniques.

### 5.4 Improved Inference Acceleration

Similar to training, the inference process of LLMs is computationally demanding, creating another bottleneck in adopting LLM4SE. While some studies have proposed dynamic inference methods to accelerate LLM inference [112, 113], comprehensive exploration in this area remains limited. For example, cascade inference strategies, which arrange small and large LLMs in a cascading manner and adaptively select suitable models rather than relying solely on large, slow LLMs for every query, could be investigated for LLM4SE. Additionally, methods such as non-autoregressive decoding [31, 37] and speculative decoding [12, 67] may also accelerate LLM inference and have potential applicability to LLM4SE [86]. Among these methods, non-autoregressive decoding speeds up LLM inference by generating all tokens in the sequence simultaneously, instead of sequentially as in auto-regressive decoding. Speculative decoding also improves the efficiency of model inference by using a smaller, faster draft model to generate text first, with the large LLM only verifying and adjusting tokens if necessary. If any token's logit from the large LLM differs substantially from the draft model's prediction, the large LLM regenerates that token. This process requires just one forward operation from the large LLM, significantly reducing computation time. We recommend that researchers systematically evaluate the effectiveness of these approaches in the context of LLM4SE and develop more efficient inference acceleration techniques tailored to these applications.

Furthermore, a recent study by Wei et al. [138] leverages code completion engines to filter out infeasible tokens generated by LLMs. By integrating lightweight static code completion engines to directly complete tokens without querying the LLM when only one valid option exists, efficiency in program repair tasks is improved. This highlights the potential of combining LLMs with existing program analysis tools to enhance both efficiency and sustainability in LLM4SE, offering another promising direction for further research.

### 5.5 Program Optimization

One area not touched in LLM4SE is optimizing the programs built for LLM inference. The inference pipeline of LLMs is typically constructed using human-written code, which may not fully exploit specific hardware features such as CPU/GPU instructions for parallel computing [76]. To address this, researchers can develop program transformation tools that automatically transform LLM inference code to take advantage of specific backend libraries or hardware features [86]. In addition, direct optimization of the LLM inference code by eliminating dead code [121] and

tuning compiler optimization flags [14] can also improve the efficiency and energy consumption of LLM4SE techniques. We recommend that the software engineering community further explore these techniques to help LLM inference achieve better efficiency and lower environmental impact in LLM4SE techniques.

Moreover, as a complementary direction to the existing work on specialized program grammars for LLMs, such as SimPy [115], we suggest that researchers also focus on developing optimization techniques for these emerging grammars to further enhance the efficiency and sustainability of LLM4SE techniques. For example, efforts to verify the correctness of transformation rules [16] and to merge or eliminate redundant rules [110] could reduce the complexity of program grammars. These optimizations would not only streamline the grammar but also improve LLMs' efficiency in code generation, contributing to more environmentally friendly LLM4SE techniques.

**Roadmap** — *Our research community can advance towards more efficient and green LLM4SE by developing comprehensive benchmarks, exploring more efficient training methods, optimizing compression and inference acceleration techniques, and enhancing program-level optimizations to reduce computational and environmental costs in LLM-based software engineering tools.*

## 6 Conclusion and Future Work

As LLMs gain traction in the software engineering community, concerns about their efficiency (i.e., computational cost, memory usage, and time consumption) and greenness (i.e., energy usage, water consumption, and carbon emissions) have escalated. This article provides a brief overview of the current landscape of LLM4SE solutions, focusing on their efficiency and greenness while highlighting the associated challenges and opportunities in this emerging field. We then sketch a future vision to provide insights into the potential benefits of adopting efficient and green LLM4SE solutions for industry, individual practitioners, and society. To realize this vision, we propose a roadmap for future research, outlining specific directions and potential solutions that the research community can pursue. With these efforts, we aim to motivate more people to join and contribute to the LLM4SE research journey, with the goal of fostering a new era of software engineering where LLM4SE solutions are not only effective, but also efficient and environmentally sustainable.

## References

[1] Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tur, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.), Association for Computational Linguistics, Online, 2655–2668. DOI: https://doi.org/10.18653/v1/2021.naacl-main.211

[2] Toufique Ahmed and Premkumar Devanbu. 2023. Few-shot training LLMs for project-specific code-summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*. ACM, New York, NY, Article 177, 5 pages. DOI: https://doi.org/10.1145/3551349.3559555

[3] Toufique Ahmed, Kunal Suresh Pai, Premkumar Devanbu, and Earl Barr. 2024. Automatic semantic augmentation of language model prompts (for code summarization). In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, Article 220, 13 pages. DOI: https://doi.org/10.1145/3597503.3639183

[4] Negar Alizadeh and Fernando Castor. 2024. Green AI: A preliminary empirical study on energy consumption in DL models across different runtime infrastructures. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI (CAIN '24)*. ACM, New York, NY, 134–139. DOI: https://doi.org/10.1145/3644815.3644967

[5] Gareth Ari Aye and Gail E. Kaiser. 2020. Sequence model design for code completion in the modern IDE. arXiv:2004.05249. Retrieved from https://arxiv.org/abs/2004.05249

[6] Shamil Ayupov and Nadezhda Chirkova. 2022. Parameter-efficient finetuning of transformers for source code. arXiv:2212.05901. Retrieved from https://arxiv.org/abs/2212.05901

[7] Lotfi Belkhir and Ahmed Elmeligi. 2018. Assessing ICT global emissions footprint: Trends to 2040 & recommendations. *Journal of Cleaner Production* 177 (2018), 448–463. DOI: https://doi.org/10.1016/j.jclepro.2017.12.239

[8] Lenz Belzner, Thomas Gabor, and Martin Wirsing. 2024. Large language model assisted software engineering: Prospects, challenges, and a case study. In *Bridging the Gap between AI and Reality*, Bernhard Steffen (Ed.), Springer Nature Switzerland, Cham, 355–374.

[9] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*. H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 1877–1901. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

[10] Coral Calero, Félix O. García, Gabriel Alberto García-Mireles, M. Ángeles Moraga, and Aurora Vizcaíno. 2024. Addressing Sustainability-IN Software Challenges. arXiv:2406.07380. Retrieved from https://arxiv.org/abs/2406.07380

[11] Saikat Chakraborty, Toufique Ahmed, Yangruibo Ding, Premkumar T. Devanbu, and Baishakhi Ray. 2022. NatGen: Generative pre-training by "naturalizing" source code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, New York, NY, 18–30. DOI: https://doi.org/10.1145/3540250.3549162

[12] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318. Retrieved from https://arxiv.org/abs/2302.01318

[13] Junkai Chen, Xing Hu, Zhenhao Li, Cuiyun Gao, Xin Xia, and David Lo. 2024. Code search is all you need? Improving code suggestions with code search. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, Article 73, 13 pages. DOI: https://doi.org/10.1145/3597503.3639085

[14] Junjie Chen, Ningxin Xu, Peiqi Chen, and Hongyu Zhang. 2021. Efficient compiler autotuning via Bayesian optimization. In *Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, 1198–1209. DOI: https://doi.org/10.1109/ICSE43902.2021.00110

[15] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from https://arxiv.org/abs/2107.03374

[16] Zheng Cheng, Massimo Tisi, and Joachim Hotonnier. 2020. Certifying a rule-based model transformation engine for proof preservation. In *Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems (MODELS '20)*. ACM, New York, NY, 297–307. DOI: https://doi.org/10.1145/3365438.3410949

[17] Nicole Choi. 2024. Customizing and fine-tuning LLMs: What you need to know — github.blog. Retrieved March 28, 2024 from https://github.blog/2024-02-28-customizing-and-fine-tuning-llms-what-you-need-to-know/

[18] Google Cloud. 2023. Sec-PaLM2. Retrieved March 26, 2024 from https://console.cloud.google.com/vertex-ai/publishers/google/model-garden/sec-palm

[19] Mariana Coutinho, Lorena Marques, Anderson Santos, Marcio Dahia, Cesar França, and Ronnie de Souza Santos. 2024. The role of generative AI in software development productivity: A pilot case study. In *Proceedings of the 1st ACM International Conference on AI-Powered Software (AIware '24)*. ACM, New York, NY, 131–138. DOI: https://doi.org/10.1145/3664646.3664773

[20] Luís Cruz, Xavier Franch Gutierrez, and Silverio Martínez-Fernández. 2024. Innovating for tomorrow: The convergence of SE and green AI. arXiv:2406.18142. Retrieved from https://arxiv.org/abs/2406.18142

[21] Cursor. 2023. Inference characteristics of llama-2. Retrieved September 29, 2024 from https://www.cursor.com/blog/llama-inference

[22] Alex de Vries. 2023. The growing energy footprint of artificial intelligence. *Joule* 7, 10 (2023), 2191–2194. DOI: https://doi.org/10.1016/j.joule.2023.09.004

[23] Edoardo Debenedetti, Giorgio Severi, Nicholas Carlini, Christopher A. Choquette-Choo, Matthew Jagielski, Milad Nasr, Eric Wallace, and Florian Tramèr. 2024. Privacy Side channels in machine learning systems. In *Proceedings of the 33rd USENIX Security Symposium (USENIX Security '24)*. USENIX Association, Philadelphia, PA, 6861–6848. Retrieved from https://www.usenix.org/conference/usenixsecurity24/presentation/debenedetti

[24] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Jill Burstein, Christy Doran, and Thamar Solorio (Eds.), ACM, Minneapolis, Minnesota, 4171–4186. DOI: https://doi.org/10.18653/v1/N19-1423

[25] Francisco Durán, Silverio Martinez-Fernandez, Matias Martinez, and Patricia Lago. 2024. Identifying architectural design decisions for achieving green ML serving. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI (CAIN '24)*. ACM, New York, NY, 18–23. DOI: https://doi.org/10.1145/3644815.3644962

[26] Brad Everman, Trevor Villwock, Dayuan Chen, Noe Soto, Oliver Zhang, and Ziliang Zong. 2023. Evaluating the Carbon impact of large language models at the inference stage. In *Proceedings of the 2023 IEEE International Performance, Computing, and Communications Conference (IPCCC)*, 150–157. DOI: https://doi.org/10.1109/IPCCC59175.2023.10253886

[27] Angela Fan, Beliz Gokkaya, Mark Harman, Mitya Lyubarskiy, Shubho Sengupta, Shin Yoo, and Jie M. Zhang. 2023. Large language models for software engineering: Survey and open problems. In *Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, 31–53. DOI: https://doi.org/10.1109/ICSE-FoSE59343.2023.00008

[28] Wenqi Fan, Yujuan Ding, Liangbo Ning, Shijie Wang, Hengyun Li, Dawei Yin, Tat-Seng Chua, and Qing Li. 2024. A survey on RAG meeting LLMs: Towards retrieval-augmented large language models. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*. ACM, New York, NY, 6491–6501. DOI: https://doi.org/10.1145/3637528.3671470

[29] Sidong Feng and Chunyang Chen. 2024. Prompting is all you need: Automated Android Bug replay with large language models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, Article 67, 13 pages. DOI: https://doi.org/10.1145/3597503.3608137

[30] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Findings of the Association for Computational Linguistics (EMNLP '20)*. Trevor Cohn, Yulan He, and Yang Liu (Eds.), Association for Computational Linguistics, Online, 1536–1547. DOI: https://doi.org/10.18653/v1/2020.findings-emnlp.139

[31] Marjan Ghazvininejad, Omer Levy, Yinhan Liu, and Luke Zettlemoyer. 2019. Mask-predict: Parallel decoding of conditional masked language models. arXiv:1904.09324. Retrieved from https://arxiv.org/abs/1904.09324

[32] GitHub. 2023. GitHub Copilot community. Retrieved October 03, 2023 from https://github.com/orgs/community/discussions/categories/copilot?discussions_q=category%3ACopilot+network

[33] GitHub. 2023. GitHub Copilot · your AI pair programmer. Retrieved March 22, 2024 from https://github.com/features/copilot/

[34] Muhammet Kürşat Görmez, Murat Yilmaz, and Paul M. Clarke. 2024. Large language models for software engineering: A systematic mapping study. In *Systems, Software and Services Process Improvement*. Murat Yilmaz, Paul Clarke, Andreas Riel, Richard Messnarz, Christian Greiner, and Thomas Peisl (Eds.), Springer Nature Switzerland, Cham, 64–79.

[35] Akhilesh Deepak Gotmare, Junnan Li, Shafiq Joty, and Steven C. H. Hoi. 2023. Efficient text-to-code retrieval with cascaded fast and slow transformer models. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM, New York, NY, 388–400. DOI: https://doi.org/10.1145/3611643.3616369

[36] Anastasiia Grishina, Max Hort, and Leon Moonen. 2023. The EarlyBIRD catches the bug: On exploiting early layers of encoder models for more efficient code classification. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM, New York, NY, 895–907. DOI: https://doi.org/10.1145/3611643.3616304

[37] Jiatao Gu, James Bradbury, Caiming Xiong, Victor O. K. Li, and Richard Socher. 2018. Non-autoregressive neural machine translation. In *International Conference on Learning Representations*. Retrieved from https://openreview.net/forum?id=B1l8BtlCb

[38] Wenchao Gu, Yanlin Wang, Lun Du, Hongyu Zhang, Shi Han, Dongmei Zhang, and Michael Lyu. 2022. Accelerating code search with deep hashing and code classification. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.), Association for Computational Linguistics, Dublin, Ireland, 2534–2544. DOI: https://doi.org/10.18653/v1/2022.acl-long.181

[39] Achim Guldner, Rabea Bender, Coral Calero, Giovanni S. Fernando, Markus Funke, Jens Gröger, Lorenz M. Hilty, Julian Hörnschemeyer, Geerd-Dietger Hoffmann, Dennis Junger, et al. 2024. Development and evaluation of a reference measurement model for assessing the resource and energy efficiency of software products and components—Green software measurement model (GSMM). *Future Generation Computer Systems* 155 (2024), 402–418. DOI: https://doi.org/10.1016/j.future.2024.01.033

[40] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.), Association for Computational Linguistics, Dublin, Ireland, 7212–7225. DOI: https://doi.org/10.18653/v1/2022.acl-long.499

[41] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2021. GraphCode{BERT}: Pre-training code representations with data flow. In *International Conference on Learning Representations*. Retrieved from https://openreview.net/forum?id=jLoC4ez43PZ

[42] Lianghong Guo, Yanlin Wang, Ensheng Shi, Wanjun Zhong, Hongyu Zhang, Jiachi Chen, Ruikai Zhang, Yuchi Ma, and Zibin Zheng. 2024. When to stop? Towards efficient code generation in LLMs with excess token prevention. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24)*. ACM, New York, NY, 1073–1085. DOI: https://doi.org/10.1145/3650212.3680343

[43] Matt Hamblen. 2023. Update: ChatGPT runs 10K nvidia training GPUs with potential for thousands more. Retrieved April 02, 2024 from https://www.fierceelectronics.com/sensors/chatgpt-runs-10k-nvidia-training-gpus-potential-thousands-more

[44] Junda He, Xin Zhou, Bowen Xu, Ting Zhang, Kisub Kim, Zhou Yang, Ferdian Thung, Ivana Clairine Irsan, and David Lo. 2024. Representation learning for Stack overflow posts: How far are we? *ACM Transactions on Software Engineering and Methodology* 33, 3, Article 69 (March 2024), 24 pages. DOI: https://doi.org/10.1145/3635711

[45] Dávid Hidvégi, Khashayar Etemadi, Sofia Bobadilla, and Martin Monperrus. 2024. CigaR: Cost-efficient program repair with LLMs. arXiv:2402.06598. Retrieved from https://arxiv.org/abs/2402.06598

[46] Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. 2022. An empirical analysis of compute-optimal large language model training. In *Advances in Neural Information Processing Systems*. S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 30016–30030. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2022/file/c1e2faff6f588870935f114ebe04a3e5-Paper-Conference.pdf

[47] Sebastian Hofstätter, Jiecao Chen, Karthik Raman, and Hamed Zamani. 2023. FiD-Light: Efficient and effective retrieval-augmented text generation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '23)*. ACM, New York, NY, 1437–1447. DOI: https://doi.org/10.1145/3539618.3591687

[48] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. 2024. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology* 33, 8 (Sept. 2024), 1–79. DOI: https://doi.org/10.1145/3695988

[49] Fan Hu, Yanlin Wang, Lun Du, Hongyu Zhang, Dongmei Zhang, and Xirong Li. 2024. Tackling long code search with splitting, encoding, and aggregating. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING '24)*. Nicoletta Calzolari, Min-Yen Kan, Veronique Hoste, Alessandro Lenci, Sakriani Sakti, and Nianwen Xue (Eds.), ELRA and ICCL, Torino, Italia, 15500–15510. Retrieved from https://aclanthology.org/2024.lrec-main.1347

[50] Qiang Hu, Yuejun Guo, Xiaofei Xie, Maxime Cordy, Lei Ma, Mike Papadakis, and Yves Le Traon. 2024. Active code learning: Benchmarking sample-efficient training of code models. *IEEE Transactions on Software Engineering* 50, 5 (2024), 1080–1095. DOI: https://doi.org/10.1109/TSE.2024.3376964

[51] Dong Huang, Yuhao Qing, Weiyi Shang, Heming Cui, and Jie M. Zhang. 2024. EffiBench: Benchmarking the efficiency of automatically generated code. arXiv:2402.02037. Retrieved from https://arxiv.org/abs/2402.02037

[52] Yizhan Huang, Yichen Li, Weibin Wu, Jianping Zhang, and Michael R. Lyu. 2024a. Your code secret belongs to me: Neural code completion tools can memorize hard-coded credentials. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 111 (July 2024), 23 pages. DOI: https://doi.org/10.1145/3660818

[53] Erik Johannes Husom, Sagar Sen, and Arda Goknil. 2024. Engineering carbon emission-aware machine learning pipelines. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI (CAIN '24)*. ACM, New York, NY, 118–128. DOI: https://doi.org/10.1145/3644815.3644943

[54] International Energy Agency (IEA). 2024. Energy system of Denmark. Retrieved September 25, 2024 from https://www.iea.org/countries/denmark

[55] Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. arXiv:2007.01282. Retrieved from https://arxiv.org/abs/2007.01282

[56] Heli Järvenpää, Patricia Lago, Justus Bogner, Grace Lewis, Henry Muccini, and Ipek Ozkaya. 2024. A synthesis of green architectural tactics for ML-enabled systems. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS '24)*. ACM, New York, NY, 130–141. DOI: https://doi.org/10.1145/3639475.3640111

[57] Nan Jiang, Kevin Liu, Thibaud Lutellier, and Lin Tan. 2023. Impact of code language models on automated program repair. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 1430–1442. DOI: https://doi.org/10.1109/ICSE48619.2023.00125

[58] Chao Jin, Zili Zhang, Xuanlin Jiang, Fangyue Liu, Xin Liu, Xuanzhe Liu, and Xin Jin. 2024. RAGCache: Efficient knowledge caching for retrieval-augmented generation. arXiv:2404.12457. Retrieved from https://arxiv.org/abs/2404.12457

[59] Matthew Jin, Syed Shahriar, Michele Tufano, Xin Shi, Shuai Lu, Neel Sundaresan, and Alexey Svyatkovskiy. 2023. InferFix: End-to-end program repair with LLMs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM, New York, NY, 1646–1656. DOI: https://doi.org/10.1145/3611643.3613892

[60] Eirini Kalliamvakou. 2022. Research: Quantifying GitHub copilot's impact on developer productivity and happiness. Retrieved September 26, 2024 from https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness/

[61] Aditya Kanade, Petros Maniatis, Gogul Balakrishnan, and Kensen Shi. 2020. Learning and evaluating contextual embedding of source code. In *Proceedings of the 37th International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 119). Hal Daumé III and Aarti Singh (Eds.), PMLR, 5110–5121. Retrieved from https://proceedings.mlr.press/v119/kanade20a.html

[62] Jai Kannan, Scott Barnett, Anj Simmons, Taylan Selvi, and Luis Cruz. 2024. Green Runner: A tool for efficient deep learning component selection. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI (CAIN '24)*. ACM, New York, NY, 112–117. DOI: https://doi.org/10.1145/3644815.3644942

[63] Ayush Kaushal, Tejas Vaidhya, and Irina Rish. 2023. LORD: Low rank decomposition of monolingual code LLMs for one-shot compression. arXiv:2309.14021. Retrieved from https://arxiv.org/abs/2309.14021

[64] Taebum Kim, Hyoungjoo Kim, Gyeong-In Yu, and Byung-Gon Chun. 2023. BPipe: Memory-balanced pipeline parallelism for training large language models. In *Proceedings of the 40th International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 202). Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.), PMLR, 16639–16653. Retrieved from https://proceedings.mlr.press/v202/kim23l.html

[65] Jennifer L. 2023. Meta to buy almost 7 million Carbon credits from aspiration — carboncredits.com. Retrieved March 28, 2024 from https://carboncredits.com/meta-to-buy-almost-7-million-carbon-credits-from-aspiration/

[66] Kif Leswing. 2023. ChatGPT and generative AI are booming, but the costs can be extraordinary. Retrieved April 02, 2024 from https://www.cnbc.com/2023/03/13/chatgpt-and-generative-ai-are-booming-but-at-a-very-expensive-price.html

[67] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning* (Proceedings of Machine Learning Research, Vol. 202). Andreas Krause, Emma Brunskill, Kyunghyun Cho, Barbara Engelhardt, Sivan Sabato, and Jonathan Scarlett (Eds.), PMLR, 19274–19286. Retrieved from https://proceedings.mlr.press/v202/leviathan23a.html

[68] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*. H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (Eds.), Vol. 33. Curran Associates, Inc., 9459–9474. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[69] Baolin Li, Yankai Jiang, Vijay Gadepally, and Devesh Tiwari. 2024. Toward sustainable GenAI using generation directives for carbon-friendly large language model inference. arxiv:2403.12900. Retrieved from https://arxiv.org/abs/2403.12900

[70] Guochang Li, Chen Zhi, Jialiang Chen, Junxiao Han, and Shuiguang Deng. 2024. Exploring parameter-efficient fine-tuning of large language model on automated program repair. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. ACM, New York, NY, 719–731. DOI: https://doi.org/10.1145/3691620.3695066

[71] Jiliang Li, Yifan Zhang, Zachary Karas, Collin McMillan, Kevin Leach, and Yu Huang. 2024b. Do machines and humans focus on similar code? Exploring explainability of large language models in code summarization. In *Proceedings of the 32nd IEEE/ACM International Conference on Program Comprehension (ICPC '24)*. ACM, New York, NY, 47–51. DOI: https://doi.org/10.1145/3643916.3644434

[72] Pengfei Li, Jianyi Yang, Mohammad A. Islam, and Shaolei Ren. 2023b. Making AI less "thirsty": Uncovering and addressing the secret water footprint of AI models. arXiv:2304.03271. Retrieved from https://arxiv.org/abs/2304.03271

[73] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: May the source be with you! arXiv:2305.06161. Retrieved from https://arxiv.org/abs/2305.06161

[74] Xiaonan Li, Yeyun Gong, Yelong Shen, Xipeng Qiu, Hang Zhang, Bolun Yao, Weizhen Qi, Daxin Jiang, Weizhu Chen, and Nan Duan. 2022. CodeRetriever: A large scale contrastive pre-training method for code search. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.), Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 2898–2910. DOI: https://doi.org/10.18653/v1/2022.emnlp-main.187

[75] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with AlphaCode. *Science* 378, 6624 (2022), 1092–1097. DOI: https://doi.org/10.1126/science.abq1158 https://www.science.org/doi/pdf/10.1126/science.abq1158

[76] Yijin Li, Jiacheng Zhao, Sun Qianqi, Haohui Mai, Lei Chen, Wanlu Cao, Yanfan Chen, Li Zhicheng, Ying LIU, Xinyuan Zhang, et al. 2023. SIRIUS: Harvesting whole-program optimization opportunities for DNNs. In *Proceedings of Machine Learning and Systems*. D. Song, M. Carbin, and T. Chen (Eds.), Vol. 5. Curan, 186–202. Retrieved from https://proceedings.mlsys.org/paper_files/paper/2023/file/3e4e24f7e055320fa54c03f6e816775f-Paper-mlsys2023.pdf

[77] Jiaxing Liu, Chaofeng Sha, and Xin Peng. 2023. An empirical study of parameter-efficient fine-tuning methods for pre-trained code models. In *Proceedings of the 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 397–408. DOI: https://doi.org/10.1109/ASE56229.2023.00125

[78] S. Liu, J. Keung, Z. Yang, F. Liu, Q. Zhou, and Y. Liao. 2024. Delving Into parameter-efficient fine-tuning in code change learning: An empirical study. In *Proceedings of the 2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE Computer Society, Los Alamitos, CA, USA, 465–476. DOI: https://doi.org/10.1109/SANER60148.2024.00055

[79] Yue Liu, Thanh Le-Cong, Ratnadira Widyasari, Chakkrit Tantithamthavorn, Li Li, Xuan-Bach D. Le, and David Lo. 2024. Refining ChatGPT-generated code: Characterizing and mitigating code quality issues. *ACM Transactions on Software Engineering and Methodology* 33, 5, Article 116 (June 2024), 26 pages. DOI: https://doi.org/10.1145/3643674

[80] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A robustly optimized BERT pretraining approach. arXiv:1907.11692. Retrieved from https://arxiv.org/abs/1907.11692

[81] David Lo. 2023. Trustworthy and synergistic artificial intelligence for software engineering: Vision and roadmaps. In *Proceedings of the 2023 IEEE/ACM International Conference on Software Engineering: Future of Software Engineering (ICSE-FoSE)*, 69–85. DOI: https://doi.org/10.1109/ICSE-FoSE59343.2023.00010

[82] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. StarCoder 2 and the Stack v2: The next generation. arXiv:2402.19173. Retrieved from https://arxiv.org/abs/2402.19173

[83] Junyi Lu, Lei Yu, Xiaojia Li, Li Yang, and Chun Zuo. 2023. LLaMA-reviewer: Advancing code review automation with large language models through parameter-efficient fine-tuning. In *Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE Computer Society, Los Alamitos, CA, USA, 647–658. DOI: https://doi.org/10.1109/ISSRE59848.2023.00026

[84] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Proceedings of the 35th Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)*. Retrieved from https://openreview.net/forum?id=6lE4dQXaUcb

[85] Alexandra Sasha Luccioni, Sylvain Viguier, and Anne-Laure Ligozat. 2023. Estimating the carbon footprint of BLOOM, a 176B parameter language model. *Journal of Machine Learning Research* 24, 253 (2023), 1–15. Retrieved from http://jmlr.org/papers/v24/23-0069.html

[86] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Hongyi Jin, Tianqi Chen, and Zhihao Jia. 2023. Towards efficient generative large language model serving: A survey from algorithms to systems. arXiv:2312.15234. Retrieved from https://arxiv.org/abs/2312.15234

[87] Erik Nijkamp, Hiroaki Hayashi, Caiming Xiong, Silvio Savarese, and Yingbo Zhou. 2023. CodeGen2: Lessons for training LLMs on programming and natural languages. arXiv:2305.02309. Retrieved from https://arxiv.org/abs/2305.02309

[88] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. CodeGen: An open large language model for code with multi-turn program synthesis. In *Proceedings of the 11th International Conference on Learning Representations*. Retrieved from https://openreview.net/forum?id=iaYcJKpY2B_

[89] Changan Niu, Chuanyi Li, Vincent Ng, Jidong Ge, Liguo Huang, and Bin Luo. 2022. SPT-code: Sequence-to-sequence pre-training for learning source code representations. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, 2006–2018. DOI: https://doi.org/10.1145/3510003.3510096

[90] Changan Niu, Chuanyi Li, Vincent Ng, David Lo, and Bin Luo. 2024. FAIR: Flow type-aware pre-training of compiler intermediate representations. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, Article 33, 12 pages. DOI: https://doi.org/10.1145/3597503.3608136

[91] Liang Niu, Shujaat Mirza, Zayd Maradni, and Christina Pöpper. 2023. CodexLeaks: Privacy leaks from code generation language models in GitHub copilot. In *Proceedings of the 32nd USENIX Security Symposium (USENIX Security '23)*. USENIX Association, Anaheim, CA, 2133–2150. Retrieved from https://www.usenix.org/conference/usenixsecurity23/presentation/niu

[92] OpenAI. 2024. GPT-4 technical report. arXiv:2303.08774. Retrieved from https://arxiv.org/abs/2303.08774

[93] K. Pal, A. Bajaj, P. Banerjee, A. Dutcher, M. Nakamura, Z. Basque, H. Gupta, S. Sawant, U. Anantheswaran, Y. Shoshitaishvili, et al. 2024. Len or index or count, anything but v1: Predicting variable names in decompilation

output with transfer learning. In *Proceedings of the 2024 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, Los Alamitos, CA, USA, 4069–4087. DOI: https://doi.org/10.1109/SP54263.2024.00152

[94] Md Rizwan Parvez, Wasi Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Retrieval augmented code generation and summarization. In *Proceedings of the Findings of the Association for Computational Linguistics (EMNLP '21)*. Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.), Association for Computational Linguistics, Punta Cana, Dominican Republic, 2719–2734. DOI: https://doi.org/10.18653/v1/2021.findings-emnlp.232

[95] David Patterson, Joseph Gonzalez, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David So, Maud Texier, and Jeff Dean. 2021. Carbon emissions and large neural network training. arXiv:2104.10350. Retrieved from https://arxiv.org/abs/2104.10350

[96] Long Phan, Hieu Tran, Daniel Le, Hieu Nguyen, James Annibal, Alec Peltekian, and Yanfang Ye. 2021. CoTexT: Multi-task learning with code-text transformer. In *Proceedings of the 1st Workshop on Natural Language Processing for Programming (NLP4Prog '21)*. Royi Lachmy, Ziyu Yao, Greg Durrett, Milos Gligoric, Junyi Jessy Li, Ray Mooney, Graham Neubig, Yu Su, Huan Sun, and Reut Tsarfaty (Eds.), Association for Computational Linguistics, Online, 40–47. DOI: https://doi.org/10.18653/v1/2021.nlp4prog-1.5

[97] Moumita Das Purba, Arpita Ghosh, Benjamin J. Radford, and Bill Chu. 2023. Software vulnerability detection using large language models. In *Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW)*, 112–119. DOI: https://doi.org/10.1109/ISSREW60843.2023.00058

[98] Mario Rodriguez. 2023. Research: Quantifying GitHub copilot's impact on code quality. Retrieved September 26, 2024 from https://github.blog/news-insights/research/research-quantifying-github-copilots-impact-on-code-quality/

[99] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Romain Sauvestre, Tal Remez, et al. 2024. Code Llama: Open foundation models for code. arXiv:2308.12950. [cs.CL] Retrieved from https://arxiv.org/abs/2308.12950

[100] Mootez Saad, José Antonio Hernández López, Boqi Chen, Dániel Varró, and Tushar Sharma. 2024. ALPINE: An adaptive language-agnostic pruning method for language models for code. arXiv:2407.04147. Retrieved from https://arxiv.org/abs/2407.04147

[101] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devesh Tiwari, and Vijay Gadepally. 2023. From words to watts: Benchmarking the energy costs of large language model inference. In *Proceedings of the 2023 IEEE High Performance Extreme Computing Conference (HPEC)*, 1–9. DOI: https://doi.org/10.1109/HPEC58863.2023.10363447

[102] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105. DOI: https://doi.org/10.1109/TSE.2023.3334955

[103] Google Cloud Security. 2023. Introducing VirusTotal code insight: Empowering threat analysis with generative AI. Retrieved March 26, 2024 from https://blog.virustotal.com/2023/04/introducing-virustotal-code-insight.html

[104] Chaoxuan Shi, Tingwei Zhu, Tian Zhang, Jun Pang, and Minxue Pan. 2023. Structural-semantics guided program simplification for understanding neural code intelligence models. In *Proceedings of the 14th Asia-Pacific Symposium on Internetware* (Internetware '23). ACM, New York, NY, 1–11. DOI: https://doi.org/10.1145/3609437.3609438

[105] Ensheng Shi, Yanlin Wang, Hongyu Zhang, Lun Du, Shi Han, Dongmei Zhang, and Hongbin Sun. 2023. Towards efficient fine-tuning of pre-trained code models: An experimental study and beyond. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '23)*. ACM, New York, NY, 39–51. DOI: https://doi.org/10.1145/3597926.3598036

[106] Jieke Shi, Zhou Yang, Junda He, Bowen Xu, and David Lo. 2022. Can identifier splitting improve open-vocabulary language model of code?. In *Proceedings of the 2022 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 1134–1138. DOI: https://doi.org/10.1109/SANER53432.2022.00130

[107] Jieke Shi, Zhou Yang, Hong Jin Kang, Bowen Xu, Junda He, and David Lo. 2024. Greening large language models of code. In *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Society (ICSE-SEIS'24)*. ACM, New York, NY, 142–153. DOI: https://doi.org/10.1145/3639475.3640097

[108] Jieke Shi, Zhou Yang, Bowen Xu, Hong Jin Kang, and David Lo. 2023. Compressing pre-trained models of code into 3 MB. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*. ACM, New York, NY, Article 24, 12 pages. DOI: https://doi.org/10.1145/3551349.3556964

[109] Jasvinder Singh. 2023. What is the energy consumption of AI enabled large language modelling (LLM'S) types of searches? — linkedin.com. Retrieved March 27, 2024 from https://www.linkedin.com/pulse/what-energy-consumption-ai-enabled-large-language-modelling-singh

[110] Dominic Steinhöfel. 2020. REFINITY to model and Prove program transformation rules. In *Programming Languages and Systems*. Bruno C. D. S. Oliveira (Ed.), Springer International Publishing, Cham, 311–319.

[111] Chia-Yi Su and Collin McMillan. 2024. Distilled GPT for source code summarization. *Automated Software Engineering* 31, 1 (2024), 22. 1573–7535 DOI: https://doi.org/10.1007/s10515-024-00421-4

[112] Zhensu Sun, Xiaoning Du, Fu Song, Shangwen Wang, and Li Li. 2024. When neural code completion models size up the situation: Attaining cheaper and faster completion through dynamic model inference. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, Article 75, 12 pages. DOI: https://doi.org/10.1145/3597503.3639120

[113] Zhensu Sun, Xiaoning Du, Fu Song, Shangwen Wang, Mingze Ni, and Li Li. 2023. Don't complete it! Preventing unhelpful code completion for productive and sustainable neural code completion systems. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering: Companion Proceedings (ICSE-Companion)*. IEEE Computer Society, Los Alamitos, CA, USA, 324–325. DOI: https://doi.org/10.1109/ICSE-Companion58688.2023.00089

[114] Zhensu Sun, Xiaoning Du, Fu Song, Shangwen Wang, Mingze Ni, Li Li, and David Lo. 2024. Don't complete it! Preventing unhelpful code completion for productive and sustainable neural code completion systems. *ACM Transactions on Software Engineering and Methodology* (Aug. 2024), 1049–331X DOI: https://doi.org/10.1145/3688831

[115] Zhensu Sun, Xiaoning Du, Zhou Yang, Li Li, and David Lo. 2024. AI coders are among us: Rethinking programming language grammar towards efficient code generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '24)*. ACM, New York, NY, 1124–1136. DOI: https://doi.org/10.1145/3650212.3680347

[116] Google Sustainability. 2023. Aiming to achieve net-zero emissions. Retrieved March 28, 2024 from https://sustainability.google/operating-sustainably/net-zero-carbon/

[117] Alexey Svyatkovskiy, Shao Kun Deng, Shengyu Fu, and Neel Sundaresan. 2020. IntelliCode compose: Code generation using transformer. In *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*. ACM, New York, NY, 1433–1443. DOI: https://doi.org/10.1145/3368089.3417058

[118] Alexey Svyatkovskiy, Sebastian Lee, Anna Hadjitofi, Maik Riechert, Juliana Vicente Franco, and Miltiadis Allamanis. 2021. Fast and memory-efficient neural code completion. In *Proceedings of the 2021 IEEE/ACM 18th International Conference on Mining Software Repositories (MSR)*, 329–340. DOI: https://doi.org/10.1109/MSR52588.2021.00045

[119] Jeniya Tabassum, Mounica Maddela, Wei Xu, and Alan Ritter. 2020. Code and named entity recognition in StackOverflow. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Dan Jurafsky, Joyce Chai, Natalie Schluter, and Joel Tetreault (Eds.), Association for Computational Linguistics, Online, 4913–4926. DOI: https://doi.org/10.18653/v1/2020.acl-main.443

[120] Chandra Thapa, Seung Ick Jang, Muhammad Ejaz Ahmed, Seyit Camtepe, Josef Pieprzyk, and Surya Nepal. 2022. Transformer-based language models for software vulnerability detection. In *Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC '22)*. ACM, New York, NY, 481–496. DOI: https://doi.org/10.1145/3564625.3567985

[121] Theodoros Theodoridis, Manuel Rigger, and Zhendong Su. 2022. Finding missed optimizations through the lens of dead code elimination. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '22)*. ACM, New York, NY, 697–709. DOI: https://doi.org/10.1145/3503222.3507764

[122] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. LLaMA: Open and efficient foundation language models. arXiv:2302.13971. Retrieved from https://arxiv.org/abs/2302.13971

[123] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288. Retrieved from https://arxiv.org/abs/2307.09288

[124] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. 2024. SynCode: LLM generation with grammar augmentation. arXiv:2403.01632. Retrieved from https://arxiv.org/abs/2403.01632

[125] Tina Vartziotis, Ippolyti Dellatolas, George Dasoulas, Maximilian Schmidt, Florian Schneider, Tim Hoffmann, Sotirios Kotsopoulos, and Michael Keckeisen. 2024. Learn to code sustainably: An empirical study on LLM-based green code generation. arXiv:2403.03344. Retrieved from https://arxiv.org/abs/2403.03344

[126] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*. I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett (Eds.), Vol. 30. Curran Associates, Inc. Retrieved from https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[127] Nalin Wadhwa, Jui Pradhan, Atharv Sonwane, Surya Prakash Sahu, Nagarajan Natarajan, Aditya Kanade, Suresh Parthasarathy, and Sriram Rajamani. 2024. CORE: Resolving code quality issues using LLMs. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 36 (July 2024), 23 pages. DOI: https://doi.org/10.1145/3643762

[128] Deze Wang, Boxing Chen, Shanshan Li, Wei Luo, Shaoliang Peng, Wei Dong, and Xiangke Liao. 2023. One adapter for all programming languages? Adapter tuning for code search and summarization. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 5–16. DOI: https://doi.org/10.1109/ICSE48619.2023.00013

[129] Deze Wang, Zhouyang Jia, Shanshan Li, Yue Yu, Yun Xiong, Wei Dong, and Xiangke Liao. 2022. Bridging pre-trained models and downstream tasks for source code understanding. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, 287–298. DOI: https://doi.org/10.1145/3510003.3510062

[130] Fang Wang, Jean Damascene Harindintwali, Zhizhang Yuan, Min Wang, Faming Wang, Sheng Li, Zhigang Yin, Lei Huang, Yuhao Fu, Lei Li, et al. 2021. Technologies and perspectives for achieving carbon neutrality. *The Innovation* 2, 4 (2021), 100180. DOI: https://doi.org/10.1016/j.xinn.2021.100180

[131] Junjie Wang, Yuchao Huang, Chunyang Chen, Zhe Liu, Song Wang, and Qing Wang. 2024. Software testing with large language models: Survey, landscape, and vision. *IEEE Transactions on Software Engineering* 50, 4 (2024), 911–936. DOI: https://doi.org/10.1109/TSE.2024.3368208

[132] Yue Wang, Hung Le, Akhilesh Deepak Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven Hoi. 2023. CodeT5+: Open code large language models for code understanding and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Retrieved from https://openreview.net/forum?id=uu6Oq7MN7g

[133] Yan Wang, Xiaoning Li, Tien N. Nguyen, Shaohua Wang, Chao Ni, and Ling Ding. 2024. Natural is the best: Model-agnostic code simplification for pre-trained large language models. *Proceedings of the ACM on Software Engineering* 1, FSE, Article 27 (July 2024), 23 pages. DOI: https://doi.org/10.1145/3643753

[134] Yue Wang, Weishi Wang, Shafiq Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.), Association for Computational Linguistics, online and Punta Cana, Dominican Republic, 8696–8708. DOI: https://doi.org/10.18653/v1/2021.emnlp-main.685

[135] Zora Zhiruo Wang, Akari Asai, Xinyan Velocity Yu, Frank F. Xu, Yiqing Xie, Graham Neubig, and Daniel Fried. 2024. CodeRAG-Bench: Can retrieval augment code generation? arXiv:2406.14497. Retrieved from https://arxiv.org/abs/2406.14497

[136] Xiaokai Wei, Sujan Kumar Gonugondla, Shiqi Wang, Wasi Ahmad, Baishakhi Ray, Haifeng Qian, Xiaopeng Li, Varun Kumar, Zijian Wang, Yuchen Tian, et al. 2023. Towards greener yet powerful code generation via quantization: An empirical study. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM, New York, NY, 224–236. DOI: https://doi.org/10.1145/3611643.3616302

[137] Yuxiang Wei, Zhe Wang, Jiawei Liu, Yifeng Ding, and Lingming Zhang. 2024. Magicoder: Empowering code generation with OSS-instruct. In *Proceedings of the 41st International Conference on Machine Learning*. Retrieved from https://openreview.net/forum?id=XUeoOBid3x

[138] Yuxiang Wei, Chunqiu Steven Xia, and Lingming Zhang. 2023. Copiloting the copilots: Fusing large language models with completion engines for automated program repair. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '23)*. ACM, New York, NY, 172–184. DOI: https://doi.org/10.1145/3611643.3616271

[139] Sarah Wells. 2023. Generative AI's energy problem today is foundational. Retrieved March 28, 2024 from https://spectrum.ieee.org/ai-energy-consumption

[140] Martin Weyssow, Xin Zhou, Kisub Kim, David Lo, and Houari Sahraoui. 2024. Exploring parameter-efficient fine-tuning techniques for code generation with large language models. arXiv:2308.10462. Retrieved from https://arxiv.org/abs/2308.10462

[141] Wikipedia. 2024. List of nvidia graphics processing units. Retrieved September 29, 2024 from https://en.wikipedia.org/wiki/List_of_Nvidia_graphics_processing_units

[142] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, 1482–1494. DOI: https://doi.org/10.1109/ICSE48619.2023.00129

[143] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming (MAPS 2022)*. ACM, New York, NY, 1–10. DOI: https://doi.org/10.1145/3520312.3534862

[144] Jingfeng Yang, Hongye Jin, Ruixiang Tang, Xiaotian Han, Qizhang Feng, Haoming Jiang, Shaochen Zhong, Bing Yin, and Xia Hu. 2024. Harnessing the power of LLMs in practice: A survey on ChatGPT and beyond. *ACM Transactions on Knowledge Discovery from Data* 18, 6, Article 160 (April 2024), 32 pages. DOI: https://doi.org/10.1145/3649506

[145] Zhou Yang, Zhensu Sun, Terry Zhuo Yue, Premkumar Devanbu, and David Lo. 2024. Robustness, security, privacy, explainability, efficiency, and usability of large language models for code. arXiv:2403.07506. Retrieved from https://arxiv.org/abs/2403.07506

[146] Zhou Yang, Zhipeng Zhao, Chenyu Wang, Jieke Shi, Dongsun Kim, Donggyun Han, and David Lo. 2024c. Unveiling memorization in code models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, Article 72, 13 pages. DOI: https://doi.org/10.1145/3597503.3639074

[147] Deborah Yao. 2023. Microsoft's GitHub copilot loses $20 a month per user. Retrieved March 28, 2024 from https://aibusiness.com/nlp/github-copilot-loses-20-a-month-per-user

[148] Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, Kurt Keutzer, and Cho-Jui Hsieh. 2020. Large batch optimization for deep learning: Training BERT in 76 minutes. In *International Conference on Learning Representations*. Retrieved from https://openreview.net/forum?id=Syx4wnEtvH

[149] Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Qianxiang Wang, and Tao Xie. 2024. CoderEval: A benchmark of pragmatic code generation with generative pre-trained models. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, Article 37, 12 pages. DOI: https://doi.org/10.1145/3597503.3623316

[150] Ye Yuan, Jiacheng Shi, Zongyao Zhang, Kaiwei Chen, Jingzhi Zhang, Vincenzo Stoico, and Ivano Malavolta. 2024. The impact of knowledge distillation on the energy consumption and runtime efficiency of NLP models. In *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI (CAIN '24)*. ACM, New York, NY, 129–133. DOI: https://doi.org/10.1145/3644815.3644966

[151] Jiyang Zhang, Sheena Panthaplackel, Pengyu Nie, Junyi Jessy Li, and Milos Gligoric. 2023. CoditT5: Pretraining for source code and natural language editing. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE '22)*. ACM, New York, NY, Article 22, 12 pages. DOI: https://doi.org/10.1145/3551349.3556955

[152] Quanjun Zhang, Chunrong Fang, Yang Xie, Yaxin Zhang, Yun Yang, Weisong Sun, Shengcheng Yu, and Zhenyu Chen. 2024. A survey on large language models for software engineering. arXiv:2312.15223. Retrieved from https://arxiv.org/abs/2312.15223

[153] Ting Zhang, Ivana Clairine Irsan, Ferdian Thung, and David Lo. 2024. CUPID: Leveraging ChatGPT for more accurate duplicate bug report detection. arXiv:2308.10022. Retrieved from https://arxiv.org/abs/2308.10022

[154] Zhaowei Zhang, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2022. Diet code is healthy: Simplifying programs for pre-trained models of code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. ACM, New York, NY, 1073–1084. DOI: https://doi.org/10.1145/3540250.3549094

[155] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. 2024. GaLore: Memory-efficient LLM training by gradient low-rank projection. arXiv:2403.03507. Retrieved from https://arxiv.org/abs/2403.03507

[156] Zibin Zheng, Kaiwen Ning, Yanlin Wang, Jingwen Zhang, Dewu Zheng, Mingxi Ye, and Jiachi Chen. 2024. A survey of large language models for code: Evolution, benchmarking, and future trends. arXiv:2311.10372. Retrieved from https://arxiv.org/abs/2311.10372

[157] Fan Zhou, Zengzhi Wang, Qian Liu, Junlong Li, and Pengfei Liu. 2024. Programming every example: Lifting pre-training data quality like experts at scale. arXiv:2409.17115. Retrieved from https://arxiv.org/abs/2409.17115

[158] Xin Zhou, Bowen Xu, DongGyun Han, Zhou Yang, Junda He, and David Lo. 2023. CCBERT: Self-supervised code change representation learning. In *Proceedings of the 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME)*, 182–193. DOI: https://doi.org/10.1109/ICSME58846.2023.00028

[159] Xin Zhou, Ting Zhang, and David Lo. 2024. Large language model for vulnerability detection: Emerging results and future directions. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER' 24)*. ACM, New York, NY, 47–51. DOI: https://doi.org/10.1145/3639476.3639762

[160] Qihao Zhu, Qingyuan Liang, Zeyu Sun, Yingfei Xiong, Lu Zhang, and Shengyu Cheng. 2024. GrammarT5: Grammar-integrated pretrained encoder-decoder neural model for code. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, New York, NY, Article 76, 13 pages. DOI: https://doi.org/10.1145/3597503.3639125

[161] Andrei Zlotchevski, Dawn Drain, Alexey Svyatkovskiy, Colin B. Clement, Neel Sundaresan, and Michele Tufano. 2022. Exploring and evaluating personalized models for code generation. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*, 1500–1508. DOI: https://doi.org/10.1145/3540250.3558959

Received 5 April 2024; revised 30 September 2024; accepted 7 November 2024